Automated Template-free Synthesis of Instruction-Centric Leakage Contracts for Black-Box CPUs

MICRO 2026 Submission #**1089** – Confidential Draft – Do NOT Distribute!!

Elvira Moreno Sánchez*
IMDEA Software Institute
Universidad Politécnica de Madrid
elvira.moreno@imdea.org

Tiziano Marinaro*
CISPA Helmholtz Center for Information Security
Saarland University
tiziano.marinaro@cispa.de

Ryan Williams
Northeastern University
ryan.williams@northeastern.edu

Marco Patrignani
University of Trento
marco.patrignani@unitn.it

Roberto Guanciale
KTH Royal Institute of Technology
roberto.guanciale@kth.se

Hamed Nemati
KTH Royal Institute of Technology
hamed.nemati@kth.se

Marco Guarnieri
IMDEA Software Institute
marco.guarnieri@imdea.org

Abstract

Side-channel attacks pose a significant security threat for modern computing platforms, because they exploit subtle discrepancies in CPU behaviors to leak sensitive information. To model the information leaked by a CPU via microarchitectural side-channels, recent work proposed *leakage contracts*: an ISA-level security abstraction that provides the foundations for secure CPU programming. Unfortunately, due to the complexity of current microarchitectures, devising a leakage contract for a CPU requires extensive manual effort and thus modern CPUs lack dedicated leakage contracts.

In this paper, we present a methodology to extract sound and precise instruction-centric leakage contracts for major CPU architectures with minimal manual intervention. We implemented this technique in MALCOS, the first template-free tool that automates the synthesis of leakage contracts for black-box CPUs. We evaluate MALCOS on x86 and ARM CPUs, and show that the contracts it synthesizes are sound and precise. Our results demonstrate that learning leakage contracts from black-box CPUs is feasible.

Keywords

Microarchitectural attacks, Leakage contracts, Program synthesis

1 Introduction

Side-channel attacks exploit variations in processor behavior—such as execution time [10, 24, 47, 49, 62] or cache access patterns [3, 28, 42, 53]—to infer sensitive information from otherwise secure software. To safeguard security-critical software, developers often adopt the Constant Time model [8]. This model assumes that the only sources of side-channel leaks are control-flow instructions and memory accesses, and secure programs need to make both secret-independent to prevent leaks. This assumption, however, is violated in modern CPUs. Processor optimizations—ranging from arithmetic optimizations for values like 0 and 1 [6] to advanced speculative execution techniques [33, 34]—can introduce new microarchitectural side-channel vulnerabilities. Hence, there is a growing need for techniques that enable developers and system architects to precisely characterize and mitigate such leaks.

Leakage contracts [27, 41] augment the Instruction Set Architecture (ISA) with a specification of all observable side-channel leaks within a CPU. This enables secure system development since programmers are made aware of the exploitable side-channels that are traditionally obscured at the ISA level. Unfortunately, constructing leakage contracts for modern CPUs is a complex task: it requires extensive reverse engineering, expert knowledge, and significant time investment [30], making it impractical to apply across the diverse landscape of commercially available CPUs.

Recently, several approaches have been proposed to synthesize *instruction-centric* contracts [18, 19, 31, 39, 59], i.e., a specific class of contracts where leaks are characterized as a function of instruction operands, which can capture subtle data-dependent instruction-level optimizations [56]. These tools reduce the manual effort needed to construct a comprehensive leakage contract for a given CPU by automating the characterization of instruction-level leaks, whereas other classes of leaks can be characterized using existing largely manual approaches [30].

However, existing contract synthesis approaches [18, 19, 31, 39, 59] suffer from two core limitations. First, they require access to a CPU's design at Register-Transfer Level (RTL) and, so far, have been applied only to small RISC-V cores. This limits their applicability to complex commercial CPUs (e.g., the latest x86 and ARM cores), where subtle microarchitectural leaks are prevalent. Second, they require a user-provided *contract template* that directly defines the leaks that can be captured by all contracts in the synthesis' search space. For instance, the template might allow capturing leaks that depend on whether "an operand's value is 0." This limits the scope of these tools since they won't be able to capture leaks that are not explicitly part of the template. E.g., a timing leak introduced by a computation simplification optimization on multiplications where the multiplication unit short-circuits whenever operands are 0 or 1 cannot be captured by a template that only allows expressing

*Both authors contributed equally to this research "an operand's value is 0." Although some works [39, 59] suggested to manually tailor the template to account for missed leaks, this is time-consuming (as it often requires tinkering with the tool's internal implementation) and significantly reduces automation.

To address these issues, we propose a *template-free* methodology to extract instruction-centric leakage contracts from black-box CPUs, which we implement in the MALCOS contract synthesizer. Since MALCOS is black-box, it can directly work with commercial CPUs for which the RTL code is not available. Furthermore, by being template-free, MALCOS overcomes the limitations of template-based approaches and it can synthesize instruction-centric contracts with minimal manual intervention. Next, we overview our contributions.

Contract synthesis methodology (§4): We propose a methodology for automatically synthesizing instruction-centric leakage contracts based on hardware observations extracted from a black-box CPU. For this, we adopt a counterexample-driven synthesis method that refines candidate contracts based on observed hardware behavior. Our methodology consists of running the following two steps until reaching a fixed point:

- *Step 1:* Given a candidate contract (initially empty) capturing the leaks observed so far, we use existing relational leakage testing tools [41, 44] to validate the contract against a black-box CPU. These tools generate test programs and run them on the underlying CPU to discover *new microarchitectural leaks* not yet captured by the candidate contract.
- *Step 2:* Newly discovered leaks are synthesized into new ISA-level contract clauses, which are added to the original contract, ensuring that the candidate captures the discovered leak. To ensure that synthesized clauses precisely characterize the leak, our synthesis approach relies on both *counterexamples* (i.e., pair of executions showcasing a leak) and *positive examples* (i.e., pair of executions that are indistinguishable for a microarchitectural attacker).

Our methodology ensures that the synthesized contract is *sound*, i.e., it captures *all* leaks exposed during testing. Furthermore, learned contracts have *high precision*, i.e., they do not unnecessarily over-approximate actual leaks.

MALCOS synthesis tool (§5): We implement our methodology in a tool called MALCOS (MicroArchitectural Leakage COntract Synthesizer). MALCOS incrementally builds instruction-centric leakage contracts that capture leaks in a given black-box CPU. While MALCOS targets `x86` and `ARM` architectures, the approach is general and can be adapted to other CPUs. To generate test cases and detect leaks (step 1), MALCOS leverages any suitable architecture-specific relational leakage testing tool; our implementation uses REVIZOR [44, 45] for `x86` and SCAM-V [11, 41] for `ARM`. Contract synthesis (step 2) is performed using the Rosette framework [51].

Differently from existing contract synthesis approaches [18, 19, 31, 39, 59], MALCOS operates without access to the processor's RTL design. Therefore, MALCOS can be applied to off-the-shelf commercial CPUs for which RTL is not available and which lack comprehensive vendor-supplied leakage specifications. Furthermore, MALCOS goes beyond prior work by synthesizing fine-grained contracts instead of simply classifying instructions as safe or unsafe like [18, 19]. Finally, it guarantees that the synthesized contracts capture *all* observed leaks (differently from Mohr et al. [39] whose contracts may miss leaks, compromising their usefulness for secure programming) and it does not require user-provided contract templates (unlike Wang et al. [59]).

Evaluation (§6): To validate our methodology, we evaluated the accuracy of contracts generated by MALCOS on two fronts.

First, using five different contract models (ranging from the simple *constant-time* [4] model to ones capturing advanced optimizations like *register file compression* [7, 56] and *silent stores suppression* [56]) as ground truth, we used MALCOS to learn leakage contracts. Our results show that MALCOS is able to synthesize contracts that capture the ground-truth leaks and that these contracts are precise and sound based on the quantitative metrics that we have defined. Second, we use MALCOS to learn leakage contracts directly from an `x86` and an `ARM` CPUs. Our results show that MALCOS can learn leakage clauses describing well-known leaks (e.g., data-cache leaks) directly from hardware measurements.

Moreover, we compare the contracts synthesized by MALCOS with those generated by state-of-the-art *template-based* white-box synthesis tools [18, 19, 31, 39, 59] using four different targets. The results show that our *template-free* approach eliminates the inflexibility of conventional methods, leading to more precise contracts than *template-based* approaches.

Summary of contributions: To summarize, this paper makes the following contributions:

- it presents a method to extract sound and precise instruction-centric leakage contracts from black-box CPUs (§4);
- it implements this methodology in MALCOS, a tool to extract leakage contracts from processors without using templates (§5);
- it evaluates MALCOS on several architectures and different contract models as ground truth (§6).

Upon acceptance, we will release MALCOS as open-source with all datasets and scripts to reproduce our experiments.

2 Overview

We now present the core aspects of MALCOS with an example.

2.1 Capturing leaks with Leakage Contracts

We start by introducing the core concepts associated with leakage contracts.

To write side-channel-free programs for a given CPU and a given microarchitectural attacker, a programmer needs to know which program executions the attacker might distinguish to ensure that no secret data is leaked.

The leakage contracts framework [27] provides a way of formally characterizing microarchitectural side-channel leaks at the ISA level, thereby enabling secure programming. In this framework, a microarchitectural attacker is modelled by mapping each (microarchitectural) execution to a *hardware trace*, i.e., the observations that an attacker can make through microarchitectural side-channels.

To capture ISA-level leaks, a leakage contract augments the ISA with a specification of the observable side-channel leaks associated with a given CPU. It maps each (architectural) program execution to a *leakage trace*, i.e., a sequence of (ISA-level) observations exposing potentially leaked information. In MALCOS, a contract is a set of clauses $cl := ex$ `IF` pr, where ex specifies what is added to the

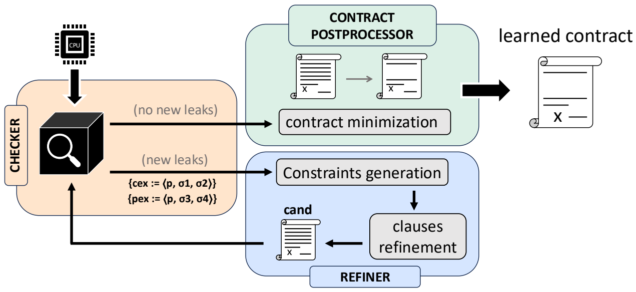

Figure 1: MALCOS contract synthesis process

leakage trace and *pr* is a predicate modeling when the clause is enabled, i.e., when the observation is added to the trace.

A CPU *satisfies* a given contract for a microarchitectural attacker Atk [27], whenever the contract captures all leaks in the underlying CPU that are observable by the attacker. That is, any two executions that result in different hardware traces, i.e., they are distinguishable by Atk, must also result in different leakage traces.

2.2 Synthesizing Leakage Contracts

Figure 1 depicts MALCOS's approach for the synthesis of leakage contracts. The approach takes as input a black-box processor Cpu and it iteratively constructs a candidate leakage contract *cand* by alternating between two phases:

(1) A *leakage testing phase* (`Checker` in Figure 1) where MALCOS attempts at finding new leaks in Cpu not captured by *cand* (capturing all leaks discovered so far).

(2) A *contract refinement phase* (`Refiner` in Figure 1) where MALCOS updates *cand* to account for a newly discovered leak.

When MALCOS cannot find new leaks, it simplifies the contract *cand* (`Postprocessor` in Figure 1), which is returned to the user. Next, we provide further details on each phase.

Leakage testing phase. The `Checker` takes a candidate contract *cand* and a CPU (treated as a black-box), and tries to discover leaks not yet captured by *cand*. Intuitively, the `Checker` executes programs and observes leaks w.r.t. a given attacker Atk. When it finds a new leak, it returns a *counterexample cex*, which is a sequence of instructions and a pair of initial states that yield the *same leakage trace* under *cand* but different hardware traces, i.e., distinguishable by Atk. It also returns *positive examples pex*, i.e., test cases indistinguishable for both the contract and the attacker.

Our synthesis methodology is `Checker`-agnostic. To emphasize its flexibility and generality, MALCOS uses two state-of-the-art relational leakage testers: Revizor [44] and Scam-V [41]. These tools differ in methodology (Revizor uses fuzzing, Scam-V symbolic execution), supported leakage sources (Revizor handles caches and timing; Scam-V supports caches), and architecture that they target (Revizor targets x86, Scam-V targets ARM).

Step 1 (A cex and a pex for RFC). *Consider a processor Cpu that implements a simple register file compression (RFC) optimization [56]. This optimization reduces the physical size of the register file by mapping all logical registers that store the value* 0 *to the same physical zero register. As pointed out in [7, 56], this optimization can result in timing leaks (due to reducing the pressure on the register file).Throughout this section we consider a microarchitectural attacker* Atk *that can precisely observe whenever RFC happens during execution.*

Starting from an empty candidate contract (`cand` $= \emptyset$*), the* `Checker` *attempts to discover a leak. For this, the* `Checker` *generates test cases, each one consisting of a program and a pair of initial states, executes them on the target CPU, and computes the hardware traces to detect potential leaks. Given that* `cand` $= \emptyset$*, the* `Checker` *discovers the test case* (p, σ_1, σ_2) *as a counterexample (*`cex`*):*

$$p := \texttt{MOV rax,rbx} \quad \sigma_1 := (\texttt{rbx} \mapsto 0) \qquad \sigma_2 := (\texttt{rbx} \mapsto 1)$$

Here, the program p *consists of an instruction assigning to register* `rax` *the value of register* `rbx`*, where the value of* `rbx` *is* 0 *in initial state* σ_1 *and a value different from* 0 *in initial state* σ_2*. Therefore, RFC happens when executing* p *from* σ_1*, but does not happen when executing* p *from* σ_2*, which results in the same leakage traces w.r.t. cand but different hardware traces for the attacker* Atk*.*

At the same time, the `Checker` *also discovers the test case* (p, σ_2, σ_3)*, where* $\sigma_3 := (\texttt{rbx} \mapsto 2)$*, as a positive example, since the two executions result in indistinguishable hardware* and *leakage traces, i.e., the attacker* Atk *cannot distinguish them.*

Contract refinement phase. The `Refiner` takes the counterexample `cex` (describing a newly-discovered leak) and the positive example `pex` (describing executions that should *not* be distinguished), and generates a new contract clause that captures the new leak. The `Refiner` discovers such a contract clause as a syntax-driven synthesis task implemented on top of the Rosette solver [51].

Positive examples are essential for *precision*. Without them, the `Refiner` may yield an over-approximate clause that exposes more information than needed (e.g., exposing *all* written register values rather than only writes of value 0). The `Refiner` ensures that the generated clause distinguish *as many counterexamples* as possible while distinguishing *as few positive examples* as possible. Finally, the `Refiner` adds the newly generated clause to the original contract to generate a new candidate contract `cand`. This process iterates, i.e., checking and refining to discover leaks not yet captured, until no further leaks are found.

Step 2 (A leakage contract for RFC). *The* `Refiner` *analyzes the counterexample* `cex` $:= \langle p, \sigma_1, \sigma_2 \rangle$ *and the positive example* `pex` $:= \langle p, \sigma_2, \sigma_3 \rangle$ *of Step 1 to generate a clause capturing the leak. It uses the Rosette solver to identify a new clause cl of the form ex* `IF` *pr that distinguishes executions of* p *from* σ_1 *and* σ_2*, while not distinguishing executions of* p *from* σ_2 *and* σ_3*. This yields the following contract* cl_1*:*

$$\begin{aligned} cl_1 \coloneqq \ \underline{\textit{post-operand-value(0)}} \ \texttt{IF} \ & [(\underline{\textit{operand-type(0)} = \texttt{reg}}) \\ \texttt{and} \ & (\underline{\textit{operand-access(0)} = \texttt{write}}) \\ \texttt{and} \ & (\underline{\textit{post-operand-value(0)} = \texttt{0}})] \end{aligned}$$

This contract exposes on the leakage trace the final value of the first operand (index 0*) whenever the first operand is a register and it is written with a value* `0`*. Crucially,* `pex` $:= \langle p, \sigma_2, \sigma_3 \rangle$*, where both states assign non-zero values to* `rbx`*, so* Atk *cannot distinguish them, prevents the* `Refiner` *from generating a coarser clause, yielding a* precise cl_1 *that only fires when RFC actually occurs.*

$$
\begin{array}{llll}
\textit{(Bitstrings)} & bs & \coloneqq & \{0,1\}^n \\
\textit{(Expressions)} & ex & \coloneqq & bs \mid \ominus ex \mid ex_1 \oplus ex_2 \mid ex[bs_1 : bs_2] \\
 & & \mid & \mathtt{REG}(bs) \mid \mathtt{OPCODE} \mid \mathtt{OP\_TYPE}(bs) \\
 & & \mid & \mathtt{OP\_ACC}(bs) \mid \mathtt{OP\_VAL}(bs) \mid \\
 & & \mid & \mathtt{POST\_REG}(bs) \mid \mathtt{POST\_OP\_VAL}(bs) \\
\textit{(Predicates)} & pr & \coloneqq & \{0,1\} \mid ex_1 = ex_2 \mid ex_1 < ex_2 \\
 & & \mid & \mathtt{NOT}\ pr \mid pr_1\ \mathtt{AND}\ pr_2 \mid pr_1\ \mathtt{OR}\ pr_2 \\
\textit{(Clause)} & cl & \coloneqq & ex\ \mathtt{IF}\ pr \\
\textit{(Contract)} & C & \coloneqq & \emptyset \mid C, cl
\end{array}
$$

Figure 2: ICL syntax

***Contract post-processing*.** Afterwards, MALCOS invokes the `Post-processor` to minimize the contract by unifying different clauses and by reducing the number of unnecessary clauses to simplify the final (`learned`) contract, further improving its precision. If CPU has no leaks beyond RFC, $\{cl_1\}$ (which exposes all registers written with a value of zero during execution) is returned as final contract.

3 Leakage Contracts

This section formalizes the syntax (§3.1) and the semantics (§3.2) of contracts, as part of a language we dub ICL, and the notion of contract satisfaction [27] (§3.3), which is used to detect counterexamples and positive examples.

3.1 ICL Syntax

The syntax of ICL is given in Figure 2. In ICL, a *contract* consists of a set of leakage clauses (using the terminology of [7, 27, 44]), where each clause describes a piece of information that might be leaked through side channels by the underlying CPU. Each clause cl consists of a conditional expression ex `IF` pr where the expression ex indicates the leaked information and the predicate pr expresses under which conditions the leak happens.

The basic type in ICL is bit strings bs, with bitvectors of length one representing booleans. Expressions ex are constructed by combining bitstrings bs with unary $\ominus ex$ and binary $ex_1 \oplus ex_2$ operators, as well as the slice operator $ex[bs_1 : bs_2]$, which extracts from ex the substring between indices bs_1 and bs_2. Additionally, expressions can refer to five pre-defined functions:

- `REG`(bs) looks up into the (logical) register file and returns the value associated with the register with index bs.
- `OPCODE` returns the opcode of the instruction.
- `OP_TYPE`(bs) returns the type, which can be one of {`reg`, `mem`, `imm`}, of the operand in position bs.
- `OP_ACC`(bs) returns the access mode, which can be one of {`r`, `w`, `rw`}, of the operand in position bs.
- `OP_VAL`(bs) returns the value of the operand in position bs. This value depends on `OP_TYPE`(bs) so that, if it is a register `reg`, the operand value will be the value of the register, if it is memory `mem`, the operand value will be the memory address it points to, and if it is an immediate `imm`, the operand value will be the value of the immediate.
- `POST_REG`(bs) and `POST_OP_VAL`(bs) returns the value of operands and registers after executing the instruction.

Finally, predicates can be boolean values, equalities and inequalities between expressions, or combinations of predicates with the standard logical operators.

3.2 ICL Semantics

The key element of interest in the semantics of leakage contracts is the *leakage trace*, i.e., the sequence of observations obtained by evaluating the clauses in the contract on *every state* in the execution. In order to collect these traces (§3.2.3), ICL requires the definition of a system semantics, which captures how the underlying system works (§3.2.1) and of a contract semantics, which dictates how the observations that constitute a leakage trace are generated (§3.2.2).

This separation of semantics is unlike prior work on leakage contracts [27], where system and contract semantics are fused in one. This separation allows ICL to be modular in the underlying system semantics, for example, it can be instantiated with an ISA semantics such as the one of μ-assembly [26] or with a speculative semantics [23].

3.2.1 System Semantics. The system semantics $\rightarrow : \Sigma \times \Sigma$ maps each state to the next one by executing one instruction in program p (written $p \vdash \sigma \rightarrow \sigma'$), where Σ is the set of all states and $\sigma \in \Sigma$ is a system state. The system semantics is left abstract except for these requirements: First, the set $\Sigma_0 \subseteq \Sigma$ denotes the set of initial states. Second, states $\sigma \in \Sigma$ contain a memory m and a register assignment r that, respectively, represent the state of memory and registers. Memories m map addresses (represented as integers) to values. Register assignments r map register identifiers (represented as bitstrings) to their values. We also assume that one of the register identifiers, denoted `PC`, represents the program counter register.

3.2.2 Contract Semantics. The semantics of an ICL contract C is obtained by extending the system semantics with observations $\delta \in Obs$ generated according to C. Formally, each ICL contract C induces a labeled semantics $\rightarrow_C : \Sigma \times Obs \times \Sigma$ specified by this inference rule:

$$
\frac{p \vdash \sigma \rightarrow \sigma' \quad \delta = \{ [\![e]\!](\sigma, \sigma') \mid e\ \mathtt{IF}\ p \in C \wedge [\![pr]\!](\sigma, \sigma') = \top \}}{p \vdash \sigma \xrightarrow{\delta}_C \sigma'}
$$

where $[\![ex]\!](\sigma, \sigma')$ (respectively, $[\![pr]\!](\sigma, \sigma')$) is the result of evaluating expression ex (respectively, predicate pr) in states σ and σ'. Note that $[\![\cdot]\!]$ needs the next-state σ' to evaluate `POST_REG`(bs) and `POST_OP_VAL`(bs). In a nutshell, the rule above labels a step of the system semantics ($p \vdash \sigma \rightarrow \sigma'$) with the set δ of observations obtained by evaluating in the current state the expressions of all leakage clauses in C whose predicates are satisfied.

3.2.3 Leakage Traces. A *leakage trace* τ is a sequence of observations obtained by applying the contract semantics $\rightarrow_C$ to an execution $p \vdash \sigma_0 \rightarrow \sigma_1 \rightarrow \ldots$, that is, $\tau = \delta_0 \delta_1 \ldots$ where $p \vdash \sigma_0 \xrightarrow{\delta_0}_C \sigma_1 \xrightarrow{\delta_1}_C \ldots$ In the following, we write $\mathtt{CTR}(C, p, \sigma)$ to denote the leakage trace (under contract C) associated with the maximal execution of program p starting from initial state σ. Furthermore, we say that two initial states σ, σ' are *C-equivalent* for a program p whenever they result in the same leakage traces. Formally: $p \vdash \sigma =_C \sigma' \stackrel{\text{def}}{=} \mathtt{CTR}(C, p, \sigma) = \mathtt{CTR}(C, p, \sigma')$.

3.3 Contract Satisfaction

Contract satisfaction precisely characterizes under which conditions a contract captures all leaks in a CPU.

Following [27, 44], we first introduce the notion of hardware traces, which capture the observational power of the microarchitectural attacker. We represent the hardware trace as the output of the function $\mathsf{HTR}\,(p, \sigma, Ctx)$, that returns the observations made by the attacker through microarchitectural side-channels. The function $\mathsf{HTR}\,(\cdot)$ takes as inputs the victim program p, the initial state σ processed by the victim program (i.e., the architectural state including registers and main memory), and the microarchitectural context Ctx in which it executes (i.e., the initial state of microarchitectural components like caches and predictors).

Informally, a CPU *satisfies a leakage contract C for a program p* [27] if any two C-equivalent states also result in identical hardware traces in any context, i.e., the executions are indistinguishable by the attacker. Formally:

$$p \vdash C \overset{\text{def}}{=} \forall \sigma, \sigma', Ctx. \\ \text{if } p \vdash \sigma =_C \sigma' \text{ then } \mathsf{HTR}\,(p, \sigma, Ctx) = \mathsf{HTR}\,(p, \sigma', Ctx)$$

We say that a CPU *satisfies a contract C* if the CPU satisfies it for every program. Dually, if a CPU does not satisfy a contract C, there is a *leakage counterexample* $\mathtt{cex} := \langle p, \sigma, \sigma' \rangle$ consisting of a program p and two initial states σ, σ' that produce different hardware traces (for some Ctx) but it generates identical leakage traces for C.

4 Synthesizing Leakage Contracts

In this section, we present our methodology for automatically synthesizing instruction-centric leakage contracts.

4.1 Synthesis algorithm

Our algorithm (shown in Algorithm 1) automatically constructs a leakage contract from a CPU under test (denoted *target* in this section) in a black-box manner. That is, the `Checker` does not have access to the CPU design, and it only interacts with the CPU by executing *test cases*, each one consisting of a program p and a sequence of initial states, to derive the corresponding hardware traces.

The synthesis algorithm takes three inputs— the CPU under test *target*, the number of test programs n used during the synthesis, and the reset parameter r—and it works in two phases.

Phase 1 spans lines 2–14, and it synthesizes a set of ICL candidate contracts that capture the leaks explored by n test programs. Phase 1 starts by computing a set P of n (randomly generated) test programs (line 2). For each program $p \in P$, the algorithm first computes the starting initial candidate contract *cand* (line 7). This candidate can be the empty contract $\emptyset$, or it can be derived from the contracts generated from prior programs. Every r iterations, the current set of candidates *cand* is added to the accumulated contract *ctr*, and the search is restarted from the empty candidate, thereby allowing MALCOS to recover from mistakes caused by low-quality examples. Next, the algorithm checks whether there are leaks in the underlying CPU under test that are not yet captured by the candidate contract *cand* (line 9, described in §4.3). For this, the algorithm relies on the `Checker` function, which returns a set of counterexamples *cexs* and a set of positive examples *pexs* for the program p. The former are leaks not yet captured by the contract

Algorithm 1: MALCOS contract synthesis approach

```
1  Procedure ContractSynthesis(target, n, r):
       // Phase 1 - synthesis
2      P ← generateSeedPrograms(n);
3      cand ← [];
4      ctr ← [];
5      CEXs ← [];
6      for p ∈ P do
7          cand, ctr ← getInitialCand(ctr, cand, r);
8          while true do
9              cexs, pexs ← Checker(p, cand, target);
10             CEXs[p] ← CEXs[p] ∪ cexs;
11             if cexs ≠ ∅ then
12                 cand ← Refiner(p, cand, cexs, pexs);
13             else
14                 break
       // Phase 2 - minimization
15     ctr ← append(ctr, cand);
16     ctr ← minimize(ctr, CEXs);
17     return ctr;
```

cand, whereas the latter represent executions that are indistinguishable for the microarchitectural attacker (which are used to improve the precision of the synthesized contract). If `Checker` finds at least one counterexample, the algorithm refines the current candidate contract *cand* to account for the newly discovered leaks using the `Refiner` function (line 12; described in §4.4). If `Checker` cannot find any further counterexamples, the algorithm terminates the testing of the current program, and moves to the next program in P.

Phase 2 spans lines 15–17, and it first consolidates the remaining candidates *cand* into the set of *ctr* contracts, then it derives the final leakage contract for the target CPU by minimizing *ctr* using the `minimize` function (line 16; described in §4.5). The minimization step allows us to remove unnecessary leakage clauses from the contract, while preserving soundness w.r.t. the explored counterexamples. That is, the final leakage contract still distinguishes all counterexamples in *CEXs*, and it is more precise (i.e., it distinguishes fewer pairs of states with the same hardware trace) than the non-minimized contract. Finally, this phase returns the final leakage contract *ctr* for the target CPU to the user.

4.2 Choosing the Initial Candidate Contract

To select the initial candidate contract when testing a new program (function `getInitialCand` on line 7 of Algorithm 1), MALCOS combines a strategy of *integration* and *resetting* based on a parameter r. Thus, the initial candidate contract *cand* for the i-th tested program is obtained by integrating the contracts synthesized for the last $i \bmod r$ programs.

This strategy allows the new candidate to account for previously found leaks–through the integration of previous contracts–and, therefore, reduces the time needed to find a candidate that covers all program leaks. At the same time, by resetting the candidate every r programs, it allows MALCOS to recover from mistakes. For instance, it allows resetting the contract when the `Checker` has not provided good enough positive examples for an individual program, which may lead the synthesis to over-approximate the leakage.

4.3 Finding Leakage Counterexamples

When learning a new leakage contract, our synthesis algorithm relies on a `Checker` that interacts with the CPU under test to determine whether the CPU satisfies the current candidate contract. The `Checker` function takes as input the current program p, the contract candidate *cand* and the target CPU *target*, and it checks whether *target* satisfies the contract *cand* w.r.t. the contract satisfaction notion from §3.3. Concretely, the `Checker` generates a *test case* for the program p (i.e., a set of initial states for p), derives the corresponding *hardware traces* and *contract traces* from *target* and *cand* respectively, and outputs a set of counterexamples and a set of positive examples.

A *counterexample* consists of two *cand*-equivalent states σ, σ' with different hardware traces, i.e., a microarchitectural attacker can distinguish the executions of p starting from σ and from σ' through a microarchitectural side-channel. That is, each counterexample represents a leak not captured by the contract *cand*.

In contrast, a *positive example* consists of two *cand*-equivalent states σ, σ' that additionally have the same hardware traces, i.e., two states that are indistinguishable for a microarchitectural attacker. Each positive example represents a pair of executions that the final contract should not distinguish. That is, if the final contract distinguishes two executions forming a positive example, the contract is an over-approximation of the desired contract (i.e., it distinguishes more executions than needed).

We remark that our synthesis algorithm is not tied to a specific implementation of `Checker`. Rather, it can work on top of any approach for checking contract satisfaction. As we discuss in §5, our implementation MALCOS instantiates `Checker` using two state-of-the-art black-box testing tools for contract satisfaction: Revizor [44] for synthesis over the x86 architecture, and Scam-V [11, 41] for synthesis over ARM architectures.

4.4 Contract Refinement

The `Refiner` function takes as input a candidate *cand*, the test program p, a set of contract counterexamples *cexs* and as set of positive examples *pexs* for p and *cand*.

In a nutshell, the function refines the current candidate contract *cand* to account for newly discovered leaks represented by the counterexamples in *cexs*. This requires finding an ICL contract *cand′* such that (a) pairs of executions that are *cand*-distinguishable are also *cand′*-distinguishable (i.e., the refined contract is not "forgetting" leaks), and (b) *cand′* distinguishes at least one counterexample in *cexs* (i.e., the refined contract indeed captures at least one newly discovered leak).

For this, we cast contract refinement as an optimization syntax-guided synthesis problem. In particular, we refine the contract *cand* by synthesizing at most m additional leakage clauses $cl_1, \ldots, cl_m$, where m is a parameter of the refinement procedure. Each clause cl_i has to satisfy the following two hard-constraints, where ? denotes a hole to be filled by the synthesis solver according to ICL's grammar:

$$cl_i = (?\ \texttt{IF}\ \texttt{REG}[\mathbf{PC}] = ? \land ?) \tag{1}$$

$$\bigvee_{(p,\sigma,\sigma') \in cexs} \texttt{CTR}\,(cl_i, p, \sigma) \neq \texttt{CTR}\,(cl_i, p, \sigma') \tag{2}$$

Hard-constraint 1 ensures that cl_i is an instruction-centric clause, which targets the leak associated with a single instruction in p (captured by the $\texttt{REG}[\mathbf{PC}] = ?$ part of the predicate, allowing us to capture fine-grained leaks that may depend on a specific instruction context). In contrast, hard-constraint 2 ensures that cl_i captures the leaks of at least one of the counterexamples *cexs* by requiring that cl_i produces distinguishable leakage observations for at least one pair of counterexample states.

Additionally, to ensure the precision of cl_i, we add the following soft-constraint, which the solver should maximize:

$$max\left(\begin{array}{l}\left|\{(p,\sigma,\sigma') \in cexs \mid \texttt{CTR}(cl_i,p,\sigma) \neq \texttt{CTR}(cl_i,p,\sigma')\}\right| \\ + \left|\{(p,\sigma,\sigma') \in pexs \mid \texttt{CTR}(cl_i,p,\sigma) = \texttt{CTR}(cl_i,p,\sigma')\}\right|\end{array}\right) \tag{3}$$

The soft-constraint 3 ensures that cl_i distinguishes most counterexamples, i.e., the one capturing more leaks. It also ensures that cl_i will be the "most precise" clause that satisfies constraints 1 and 2, i.e., the one that distinguishes as few positive examples as possible.

The synthesis solver outputs a set of instruction-centric clauses cl_i that are specific for the program p. These clauses are of the form $ex\ \texttt{IF}\ \texttt{REG}[\mathbf{PC}] = val \land pr$ where val is the address of one of p's instructions. That is, they refer explicitly to the context of the program p. Then, the `Refiner` generalizes these clauses, allowing them to be applied across all programs. Concretely, it replaces each clause $ex\ \texttt{IF}\ \texttt{REG}[\mathbf{PC}] = val \land pr$ with an equivalent clause $ex\ \texttt{IF}\ type_{p,\mathrm{PC}=val} \land pr$ where $type_{p,\mathrm{PC}=val}$ is a meta-level predicate matching the instruction type of the corresponding instruction at program counter val in p. Thus, $type_{p,\mathrm{PC}=val}$ is defined in ICL as the conjunction of predicates capturing the instruction type characteristics at the specific program counter val (namely $\texttt{OP\_TYPE}(bs)$, $\texttt{OP\_ACC}(bs)$, and $\texttt{OPCODE}(bs)$). That is, if the clause cl is associated with program counter $addr$ pointing to instruction i, the corresponding generalized clause is obtained by replacing the sub-predicate $\texttt{REG}[\mathbf{PC}] = addr$ with a predicate that is satisfied whenever the program counter points to an instruction of the same type as i (i.e., same op-code and operand types).

Finally, the `Refiner` outputs the new candidate contract *cand′* as the union of *cand* and the new clauses $cl_1, \ldots, cl_m$. This refinement satisfies points (a) and (b) above (the former follows from $cand' = cand \cup \{cl_1, \ldots, cl_m\}$ and the latter from the second constraint).

4.5 Minimization

Phase 2 of our algorithm post-processes the ICL contract to obtain a more precise one. Thus, the `minimize` function removes unnecessary clauses from the contract candidate *cand*. For this, *cand*'s clauses are sorted in terms of their precision (from higher to lower) over the test cases explored during Phase 1, where more precise clauses distinguish less attacker-equivalent pairs of executions. Then, the clauses that do not contribute to distinguishing the collected counterexamples are iteratively removed, i.e., a clause is removed if all counterexamples are still distinguishable by a contract containing only the remaining clauses. This step allows us to remove "useless" clauses that do not capture actual leaks and only reduce precision. This minimization step significantly contributes to improving the precision of the final contract (see §6.4).

5 Implementation

This section presents the MALCOS contract synthesis tool. To show the generality of our synthesis approach, we implemented two separate backends to support x86 and ARM architectures. We refer to MALCOS with each backend as MALCOS-x86 and MALCOS-ARM. Both follow the workflow outlined in Algorithm 1, but they differ in: the contract satisfaction tool (§5.1), the way contracts are refined (§5.2), and contract post-processing (§5.3). We remark that the differences between the two backends ultimately stem from the different contract satisfaction tools used.

5.1 Code Examples and Contract Satisfaction

The `Checker` part of MALCOS must generate test programs (function `generateSeedPrograms` in Algorithm 1) and extract counter- and positive examples for a given contract candidate. MALCOS implements the `Checker` with tools for relational contract testing: REVIZOR [44] for x86 and SCAM-V [41] for ARM. Both tools take as input a contract and try to discover contract violations (i.e., counterexamples) in the CPU under test by generating random programs and inputs, executing them on the actual CPU and at contract-level, and comparing collected traces. Off-the-shelf, both tools only output counterexamples, so we modified them to also extract positive examples. Each call to the `Checker` produces a single counterexample in MALCOS-x86 but yields multiple counterexamples in MALCOS-ARM.

MALCOS-x86: For x86, we rely on REVIZOR, which simulates test programs using the Unicorn CPU emulator extended with support for contracts. In MALCOS-x86, we extend REVIZOR to support ICL contracts by compiling them into event-handlers for the Unicorn emulator that computes leakage observations during execution [7].

Given that REVIZOR uses randomly-generated programs and inputs during the testing process, we use two optimizations implemented in REVIZOR—program minimization and data minimization—to tame randomness and improve the precision of the synthesized contracts. *Program minimization* minimizes test programs that result in violations. For this, whenever REVIZOR finds a counterexample for a program p, REVIZOR iteratively removes instructions from p so long as it can still find a counterexample in the smaller program p'. *Data minimization*, instead, identifies which parts of the initial states lead to a violation. For this, whenever REVIZOR finds two architectural states σ and σ' causing a violation, it iteratively modifies σ' by copying part of the state from σ (by copying registers and memory at a byte-granularity) until the violation persists. We remark that MALCOS-x86 applies these minimization steps automatically whenever the `Checker` finds a counterexample, and the minimized counterexample is then used for synthesis.

MALCOS-ARM: For ARM, we use SCAM-V as `Checker`, which combines techniques from program verification and fuzzing to perform relational testing to validate the candidate contracts. For a generated test program p, SCAM-V uses *symbolic execution* to synthesize a relation that identifies which states are observationally equivalent according to the contract being validated. Next, SCAM-V generates an instance of this relation in terms of two input states. Finally, similar to REVIZOR, SCAM-V runs the generated program with two inputs on hardware to collect hardware traces.

5.2 Contract Refinement

To refine the candidate contract (function `Refiner` in Algorithm 1), we implement the instruction-centric clause synthesis process described in §4.4 using Rosette [51], a solver-aided language extending Racket. We first formalize ICL's syntax and semantics in Rosette, then collect all state sequences from counterexamples *cexs* and positive examples *pexs* to instantiate the synthesis constraints 1–3. Rosette then uses the Z3 SMT solver [16] to synthesize ICL clauses that satisfy these constraints.

The main difference between the MALCOS-x86 and MALCOS-ARM toolchains is that MALCOS-x86 synthesizes one leakage clause per instruction in the program under test p during refinement, while MALCOS-ARM synthesizes a single clause for the entire program. This difference is due to the program minimization step in MALCOS-x86 (§5.1), which pinpoints *all* instructions involved in the leak, enabling clause synthesis for each instruction. In contrast, MALCOS-ARM does not minimize the test program, which may include irrelevant instructions. By synthesizing a single clause, we allow the solver to identify leak sources without unnecessary approximations from clauses tied to unrelated instructions.

5.3 Contract Post-Processing

Both MALCOS-x86 and MALCOS-ARM minimize the final ICL contract into a more precise one using the minimization step from §4.5. Thus, both toolchains sort clauses by the number of positive examples they (unnecessarily) distinguish, ordering from those that distinguish less (i.e., more precise clauses) to those that distinguish more. This is evaluated using the notion of C-equivalence (§3.2.3), where we say a clause cl_2 is at least as precise as cl_1 if $\forall(p, \sigma, \sigma').\ p \vdash \sigma =_{cl_2} \sigma'$ then $p \vdash \sigma =_{cl_1} \sigma'$. This implication allows us to filter out redundant or less precise clauses. The key difference between the toolchains resides on the domain in which this condition is verified, which ultimately stems from the different checkers used in each case. MALCOS-ARM uses an SMT solver to formally verify the formula above (i.e., the universal quantification ranges over all possible pairs of states in the program), whereas MALCOS-x86 relies on testing the condition on the set of known counterexamples ($\forall(p, \sigma, \sigma') \in CEXs$).

6 Evaluation

In our evaluation, we address the following questions:

- **RQ1:** How good are MALCOS' contracts?
- **RQ2:** What is the impact of contract minimization?
- **RQ3:** Can MALCOS synthesize contracts from hardware?
- **RQ4:** How do MALCOS-learned contracts compare to *template-based* approaches?

We begin by introducing the metrics used to evaluate contract quality (§6.1), then we describe the target leakage contracts used as ground truth in **RQ1**—**RQ2** (§6.2). Finally, we answer **RQ1**–**RQ4** in §6.3–6.6 respectively.

6.1 Metrics

Leakage contracts act as binary classifiers over pairs of executions, i.e., a contract indicates whether an observer can distinguish two executions. To evaluate the quality of the learned contracts, we use two standard metrics for binary classification: precision and soundness. These metrics are defined w.r.t. a *validation set* V consisting of test cases (p, σ, σ') where p is a program and σ, σ' are initial states.

Precision measures how precisely the learned contract reflects leaks in the target system. Following Mohr et al. [39] and Wang et al. [59], precision is defined as: $\frac{TP}{TP+FP}$, where TP is the number of true positives, i.e., test cases in V that are distinguishable using both contract and hardware traces, and FP is the number of false positives, i.e., test cases in V that are distinguishable only by the contract traces but not by the hardware traces.

Soundness measures the correctness of the learned contract. We define it as $\frac{TP}{TP+FN}$, where FN is the number of false negatives, i.e., test cases in V that are distinguishable by hardware traces but not by the contract traces.

6.2 Contract Models

To answer **RQ1**—**RQ2**, we use MALCOS to synthesize leakage contracts against a set of fixed contract models used as ground truth, i.e., in MALCOS these models are used as *target* to obtain hardware traces.[1] These models, which we describe next, encapsulate representative microarchitectural behaviors and distinguish relevant leakage scenarios.

- **Constant-time (ct)** models the constant-time observer commonly used when reasoning about side channels in cryptographic algorithms [4]. **ct** exposes the value of the program counter and the addresses of load and store operations throughout the execution.
- **Tag Index (TagIdx)** differs from **ct** in that it exposes only the tag and set index of memory accesses (instead of the whole memory address like in **ct**). **TagIdx** is often used to reason about the security of cryptographic implementations against cache-based side-channel attacks [22].
- **Register file compression (RFC)** models the leaks induced by the compression optimization described in §2.
- **Silent Store (SilStore)** differs from **RFC** in that it produces a leak when a value 0 is written to memory (instead of a register like **RFC**). **SilStore** models leaks from store operations that do not alter memory contents [56], and its effects have been observed on several Intel CPUs [21].
- **Multiplication simplification (mul)** models leaks caused by computation simplification over multiplications [56]. Specifically, **mul** exposes if a multiplication's operands are 0 or 1.

6.3 RQ1: How Good Are MALCOS' Contracts?

RQ1 focuses on evaluating the contracts generated by MALCOS as the final output of Algorithm 1.

[1] MALCOS-X86 and MALCOS-ARM have been extended to support a leakage contract C as synthesis target. For this, the hardware trace associated with program p and state σ is computed by simulating the program using and ISA simulator and, for each instruction, all clauses in C are evaluated w.r.t. the current architectural state to compute the corresponding leakage observations. We remark that, computing the hardware trace when using a given contract as synthesis target involve no hardware measurements, and the hardware trace consists only of the leakage observations computed by the ISA simulator.

Contract	MALCOS-X86 with Ctr. minimization			MALCOS-ARM with Ctr.minimization		
	avg. P/S	avg. Time	avg. # Clauses	P/S	Time	# Clauses
ct	1.0/1.0	11h 57m	6	1.0/1.0	5h 40m	2
TagIdx	0.79/1.0	18h 57m	14.4	1.0/1.0	13h 42m	2
RFC	1.0/1.0	10h 32m	9.8	1.0/1.0	12h 51m	6
mul	1.0/1.0	24h 21m	15.7	1.0/1.0	22h 34m	4
SilStore	1.0/1.0	1h 23m	1	1.0/1.0	5h 29m	1

Table 1: MALCOS results: Precision (P), Soundness (S), and Number of Clauses (# Clauses).

Experimental setup: We use MALCOS-X86 and MALCOS-ARM to synthesize contracts against the ground-truth models from §6.2.

For MALCOS-X86, for each target contract, we randomly generate 10000 programs and 100 random states per program, resetting the contract after every 500 programs (i.e., $R = 500$), and we use MALCOS-X86 to synthesize the contracts (using 100 positive examples during synthesis). We repeat the experiment outlined above 10 times, with different initial randomness seeds. We then measure MALCOS runtime and the precision/soundness of the learned contracts against a validation set of 50000 new random programs and 100 new random states per program (i.e., 100^2 pairs of traces), and report the average of both metrics.

For MALCOS-ARM, for each target contract, we randomly generate 100 programs, with 20 positive examples (cached at line 9 in Algorithm 1 and reused among the while loop iterations of the program under test) and 10 counterexamples, resetting the contract after every program (i.e., $R = 1$). We then measure MALCOS runtime and the precision/soundness of each synthesized contract against a validation set consisting of 100 new programs, each with 100 pairs of states (50 target-distinguishable pairs and 50 target-indistinguishable pairs).

For both MALCOS-X86 and MALCOS-ARM, we report the number of clauses in the synthesized contracts, i.e., the final number of clauses after the postprocessing phase in Algorithm 1.

Results: Table 1 summarizes our results. Both toolchains successfully learn contracts that capture all target leaks, i.e., soundness is always 100%. Both tools resolved the over-approximations during the minimization process, achieving 100% precision in almost all cases (see Table 2 for comparison).

We manually inspected the case where MALCOS-X86 could not achieve 100% precision (**TagIdx**, 79%), and we observed conservative over-approximations that justify not reaching 100% precision, confirming that the lower precision is due to MALCOS-X86 exposing more address bits than are actually needed in the final contract. We also manually inspected all synthesized contracts that achieved 100% precision (for MALCOS-X86 and MALCOS-ARM) and confirmed their equivalence to the ground truth. In particular, for MALCOS-ARM, we leveraged SCAM-V's SMT-backend to verify that, indeed, synthesized contracts and target ones are equivalent.

Contract	MALCOS-x86 without Ctr. minimization avg. P/S	avg. Time	avg. # Clauses	MALCOS-ARM without Ctr.minimization P/S	Time	# Clauses
ct	0.70/1.0	10h 31m	288.2	0.51/1.0	5h 21m	24
TagIdx	0.14/1.0	14h 14m	388.9	0.50/1.0	12h 44m	204
RFC	0.30/1.0	8h 40m	368.8	0.50/1.0	11h 36m	271
mul	0.20/1.0	14h 58m	255.5	0.50/1.0	18h 20m	404
SilStore	0.13/1.0	1h 19m	32.2	0.58/1.0	5h 9m	86

Table 2: Precision (P), Soundness (S), and Number of Clauses (# Clauses) without contract minimization.

6.4 RQ2: What is the impact of contract minimization?

RQ2 focuses on evaluating the impact of contract minimization (line 16 in Algorithm 1) on the quality of the synthesized contracts.

Experimental setup: To assess the impact of contract minimization, we repeat the experiments of §6.3 while disabling the final contract minimization phase for both MALCOS-x86 and MALCOS-ARM. As before, we measure the average MALCOS runtime, the precision/-soundness of the synthesized contracts against the same validation sets as in §6.3, and the number of clauses in the synthesized contracts (without minimization).

Results: Table 2 reports the results of our experiment. Comparing the results in Table 2 with those in Table 1 (where the synthesized contracts have been minimized) shows that, without minimization, MALCOS synthesizes contracts that are significanlty less precise, even though soundness stays at 100%. That is, minimization indeed is useful and improves the precision of synthesized contracts.

These results can be understood more clearly when comparing the number of clauses in the synthesized contracts with and without minimization (cf. columns *# Clauses* in Tables 1 and 2). Without minimization, the final contracts contain many more clauses that might unnecessarily introduce over-approximations (thereby resulting in less precise contracts) without contributing to soundness.

Finally, as expected, removing the minimization step also reduces the total execution time (since the synthesis algorithm in Algorithm 1 can directly terminate at line 15). This impact is more noticeable in more complex contracts, such as **TagIdx** and **mul**, whereas the impact is very limited or absent for simpler contracts like **ct** and **SilStore**.

6.5 RQ3: Can MALCOS Synthesize Leakage Contracts From Actual Hardware?

RQ3 focuses on whether MALCOS can be used to learn contracts from an x86 (§6.5.1) and an ARM (§6.5.2) CPUs.

6.5.1 x86.

Using MALCOS, we synthesize contracts for 13 subsets of the x86 ISA. For these subsets, we re-use REVIZOR's initial configuration from [45] where (a) all subsets include the same 8 base arithmetic/logic instructions (including their versions with memory operands), which are needed by REVIZOR to work properly, and (b) each subset extends the base instructions with unique subset-specific instructions. ?? provides a description of the subsets.

Experimental setup: We target an Intel i5-6500 CPU, which is based on the Skylake microarchitecture. For each ISA subset, we use MALCOS-x86 to synthesize the contract capturing the leaks associated with the subset's instructions. Each subset was tested with 100000 randomly generated programs, and each program executed with 100 inputs, restarting from an empty candidate every 500 programs (i.e., $R = 500$), and we used up to 100 positive examples for synthesis. To account for speculatively executed instructions, we configure REVIZOR to explore at ISA-level also mispredicted branches[2] (to account for branch speculation) and to disable store-bypass speculation (by enabling the `ssbd` patch [32]). To derive the hardware traces, we configure REVIZOR to use the `TSC` executor mode [37], where traces are obtained by measuring the test cases' execution time with `RDTSCP`.

Results: MALCOS-x86 performed the 130 million test executions, over the 13 subsets of the x86 ISA, in 17 hours[3]. Moreover, MALCOS-x86 spent 1 hour and 24 minutes for contract refinement across all subsets, and 5 minutes for contracts minimization. Table 3 summarizes the results of the synthesis campaign, and it highlights the types of synthesized clauses for each subset. We manually inspected the synthesized contracts and highlight the following findings:

- On average, the amount of attacker indistinguishable pairs of states explored during the testing campaign with respect to the synthesized contracts is only 8%.
- For all subsets, MALCOS-x86 synthesized clauses exposing the addresses of memory accesses, which capture leaks through the data cache. Even for subsets like `nop`, the base instructions might still result in memory operations. E.g., MALCOS-x86 synthesized the following clause that exposes the address of a memory load/store:

$$cl_{mem} \coloneqq \underline{\mathit{OP\_VAL(0)}}\ \texttt{IF}\ [(\underline{\mathit{OP\_TYPE(0)}\ \texttt{= mem}})\ \texttt{and}\ (\underline{\mathit{OP\_ACC(0)}\ \texttt{= r/w}})]$$

- For the `cond` subset (the only one including control-flow statements), MALCOS-x86 synthesized clauses exposing the program counter, thereby exposing control-flow leaks. For instance, MALCOS-x86 synthesized the cluase cl_{pc} below that exposes the program counter after the execution of a control-flow statement:

$$cl_{pc} \coloneqq \underline{\mathit{POST\_REG}(\boldsymbol{PC})}\ \texttt{IF}\ (\underline{\mathit{OPCODE}\ \texttt{= jmp}})$$

- For the `dmul` subset , MALCOS-x86 synthesizes clauses exposing information about the divisor operand[4]. This is consistent with prior findings showing that division operations are not constant-time [50]. For instance, MALCOS-x86 synthesized the clause cl_{div} exposing the dividend whenever the divisor is zero:

$$cl_{div} \coloneqq \underline{\mathit{OP\_VAL(1)}}\ \texttt{IF}\ [(\mathit{OPCODE}\ \texttt{= div})\ \texttt{and}\ (\underline{\mathit{OP\_VAL(0)}\ \texttt{= 0}})]$$

- For the *strn* subset (which includes `rep` instructions), MALCOS-x86 synthesized clauses exposing the `rep` counter which determines

[2]For this, we enabled the `cond` execution clause in Revizor, namely, using the always-mispredict speculative semantics [26] as the system semantics → for MALCOS (§3.2).
[3]Observe that during the 17 hours testing, the throughput of MALCOS is aligned with standard evaluations of REVIZOR [45]
[4]Note that, in order to explore interesting cases, we set for this specific subset a lower-than-usual value for the entropy generation of the states so that there is a higher probability of operands equal to 0 appearing in the tests.

x86-64 Subset	Mem. addr.	Ctrl. flow	`rep` counter	Div. by 0
`cond`: Conditional branches	✓	✓	×	×
`strn`: String operations	✓	×	✓	×
`dmul`: Division and mult.	✓	×	×	✓
`flag`: Operations on flags	✓	×	×	×
`lock`: Atomics w/ LOCK	✓	×	×	×
`atom`: Atomics w/o LOCK	✓	×	×	×
`dxfr`: Data transf. (load/store)	✓	×	×	×
`setc`: Conditional byte set	✓	×	×	×
`nop`: NOP instructions	✓	×	×	×
`logi`: Logical operations	✓	×	×	×
`conv`: Data type conversion	✓	×	×	×
`cmov`: Conditional moves	✓	×	×	×
`bit`: Bit test and bit scan	✓	×	×	×

Table 3: Summary of the x86-64 synthesis campaign across 13 ISA subsets. ✓ indicates that MALCOS synthesized clauses capturing the corresponding leakage source; × otherwise.

how many times an instruction is repeated. As an example, consider the following clause synthesized by MALCOS-x86:

$$cl_{repe} \coloneqq \underline{\mathit{OP\_VAL(1)}}\ \texttt{IF}\ [(\mathit{OPCODE} = \texttt{repe stosd})\ \texttt{and}\ (\underline{\mathit{POST\_OP\_VAL(2)} = \texttt{0}})]$$

Here, cl_{repe} exposes the destination pointer when the `rep` counter is zero, i.e., in the final iteration.

Overall, the synthesized contracts are aligned with the community's understanding of instruction-level leaks in the target CPU.

6.5.2 ARM. Using MALCOS, we synthesize a contract associated with ARM memory instructions. We analyzed 30 relevant load/store instructions, including word, (signed) byte, (signed) halfword, unscaled offsets, and register pairs (see **??** for the list of analyzed instructions).

Experimental setup: We used MALCOS-ARM to synthesize a contract on a Raspberry Pi4 with a Cortex-A72 CPU. To derive hardware traces, we configured SCAM-V to monitor the L1 cache state immediately after each test case execution, in an access-driven attack fashion [28]. SCAM-V uses a *platform module* that runs in ARM TrustZone to configure page-table attributes to control memory cacheability, clear the caches before execution, insert memory barriers around the test case code, and read the cache state using privileged debug instructions. Each experiment was repeated 10 times, and the final cache state was checked for discrepancies.

MALCOS-ARM tested each instruction individually. For each contract refinement iteration, we used up to 10 counterexamples and 20 positive examples for synthesis.

Results: The synthesis campaign took approximately 5.5 hours, of which 4 minutes were spent on contract refinement, 11 minutes on contract minimization, and the remaining time on generating counterexamples and positive examples. MALCOS-ARM produced a minimized contract consisting of six clauses. Two clauses expose the cache tag (all bits above bit 12) and index (bits 6–12) associated with individual memory loads and stores.

$$cl_{mem} \coloneqq \underline{(\mathit{OP\_VAL(1)}[64{:}6])}\ \texttt{IF}\ (\underline{\mathit{OPCODE} = \texttt{ld*/st*}})$$

MALCOS-ARM successfully captures the leakage of accessed cache lines. The least significant 6 bits represent the address offset and can be ignored when identifying the accessed cache line.

The remaining four clauses are associated with `LDP`, `LDPSW`, and `STP`, which access two 32-bit words or two 64-bit doublewords from memory at the same time. Similarly to cl_{mem}, these clauses again expose the tag and index bits of the accessed memory addresses.

6.5.3 Summary. These results confirm that MALCOS can synthesize contracts from black-box CPUs. Since MALCOS relies on black-box learning, the quality of generated contracts is inherently tied to the quality of the `Checker`'s test cases, and comprehensively identifying all leaks may require more tests. For instance, Cortex-A53 can leak the cache line offset accessed by memory loads [41], but we did not observe this in our synthesis campaign. Improving existing `Checkers` is, however, beyond the scope of our work.

6.6 RQ4: How do MALCOS-learned contracts compare to *template-based* approaches?

Previous work [18, 31, 39, 59] have developed contract synthesis tools based on templates and white-box methods which require (1) access to the processor's RTL design and (2) a set of user-provided clause templates directly defining the synthesis search space. Our work overcomes these dependencies by introducing the first black-box synthesis approach for leakage contracts that is *template-free*. Since no existing tool operates under the same black-box, *template-free* conditions as MALCOS, we cannot compare against them directly on the same hardware targets (note that [18, 31, 39, 59] target small open-source RISC-V cores whereas MALCOS focuses on x86 and ARM commercial CPUs). Instead, we design a controlled evaluation that measures the *precision* of contracts synthesized by MALCOS against the same search space as each competing technique.

Experimental setup: To enable a fair comparison, we define a set of ground-truth models as synthesis targets. This allows us to do the comparison without requiring RTL access.

We define four ground-truth models inspired by the leakage profiles of different variants of the RISC-V **Ibex** core [1] (which is the CPU targeted by the competing *template-based* tools [18, 31, 39, 59]), and of some optimizations covering representative microarchitectural behaviors and relevant leakage scenarios (like those in §6.2): (1) **Ibex-small**: the default "small" configuration of **Ibex** with constant-time multiplication (three cycles) and without caches (which leaks information about the outcome of branch instructions and accessed memory addresses). (2) **Ibex-mult-div**: **Ibex-small** extended with a non-constant-time multiplication unit [2] whose execution time depends on the operand values and it additionally leaks whether divisions and multiplications by zero happen. (3) **Ibex-optimizations**: **Ibex-mult-div** extended with the leaks captured by the contract models defined in §6.2. (4) **Ibex-slice**: **Ibex-mult-div** extended with memory leaks that expose only the tag and set index of memory accesses (**TagIdx** from §6.2), and arithmetic leaks whose execution time depends on the sign (e.g., divisions) or size (e.g., multiplications) of the operands. Note that **Ibex** is a RISC-V core, whose ISA is not supported by MALCOS. For

	MALCOS-X86	LeaSyn [59]	RTL2μPath [31]	VeloCT [18]	MALCOS-ARM	LeaSyn [59]	RTL2μPath [31]	VeloCT [18]
Ibex-small	1.0	1.0	0.99	0.99	1.0	1.0	1.0	1.0
Ibex-mult-div	1.0	1.0	0.68	0.68	1.0	1.0	0.91	0.90
Ibex-optimizations	0.89	0.57	0.37	0.32	0.92	0.56	0.54	0.54
Ibex-slice	0.73	0.41	0.36	0.18	0.89	0.63	0.57	0.57

Table 4: Comparison of precision with templates from previous works in both tools, MALCOS-X86 and MALCOS-ARM.

this reason, we instantiate the leakage profiles above for the x86 and ARM ISA (rather than for RISC-V).

We compare MALCOS against LeaSyn [59] (which extends Mohr et al. [39]), RTL2μPath [31], and VeloCT [19] (which extends ConjunCT [18]), which are the most recent and relevant *template-based* white-box synthesis tools for leakage contracts. To allow the comparison, we emulate their behavior by restricting the synthesis grammar to the clause templates supported by each technique. We also run MALCOS in its original unrestricted grammar mode as an additional point of comparison. To have a fair comparison among the tools and MALCOS, we run the evaluation in both, MALCOS-X86 and MALCOS-ARM, with the same experimental setup as in §6.3. This is necessary to account for the fact that the contract synthesized using the same model may vary depending on the specific `Checker` used. We then measure the precision of the synthesized contracts, and report the results.

Results: Table 4 summarizes the results of our analysis. For the simplest **Ibex-small** model, all templates are sufficient to synthesize precise contracts. However, when considering more complex leakage profiles, restrictive templates cannot precisely express the leaks as part of a contract. In particular, VeloCT (which only supports a template exposing all operands of an instruction) and RTL2μPath (which only supports a template exposing a subset of all operands of an instruction) already fail in precisely capturing the leaks of **Ibex-mult-div** since they cannot precisely capture the specific values of the operands, e.g., *OP_VAL(0)*=`0`. LeaSyn supports more complex templates that have been hand-crafted to capture the leaks in existing **Ibex** CPUs [39] and, thus, its template can still precisely capture the leaks in **Ibex-mult-div**. As soon as the templates become too restrictive, precision drops significantly. MALCOS can precisely capture the leaks in **Ibex-small** and **Ibex-mult-div**. Moreover, due to its template-free nature, it always results in more precise contracts than those associated with the templates supported by existing tools in complex targets like **Ibex-optimizations** and **Ibex-slice**.

7 Discussion

Limitations: MALCOS is limited by several design choices. First, it focuses on instruction-centric contracts, synthesizing only leak clauses (what is leaked) according to the terminology of [7, 27, 44]. MALCOS also cannot represent stateful clauses that capture leaks across multiple instructions (e.g., those from computation reuse or operand packing [7, 56]). Finally, MALCOS explores only user-mode code, as our `Checkers` do not support privileged instructions, excluding kernel-level leaks (e.g., page table accesses).

Black-box synthesis: MALCOS demonstrates that leakage contract synthesis for off-the-shelf CPUs is feasible without RTL access. RTL-based approaches [18, 19, 31, 39, 59] can provide more exhaustive contracts but require having access to the processor's design and manual effort, limiting their applicability. In contrast, MALCOS guarantees *bounded* soundness (all detected leaks are captured), using test cases to approximate precision. This makes it practical for commercial CPUs lacking RTL or vendor-supplied specifications.

False negatives: The synthesized contracts capture only those leaks that are exercised by the test cases and detected by the `Checker`. Leaks outside these conditions result in *false negatives*[5].

False positives: Despite minimization (§4), synthesized contracts may still over-approximate actual leaks, resulting in *false positives*. This occurs if the `Postprocessor` cannot remove all over-approximations, or if some leaks (e.g., when only a hash of a register leaks) are inherently hard to infer precisely from test cases alone.

Threats to validity of the evaluation: To cover a large space of test cases and to reduce biases introduced by manually selected test-cases, we decided to construct the validation sets using the random program generators included in the two relational testing tools supported by MALCOS, that is, Revizor and Scam-V. In particular, for MALCOS-X86, on average, more than 70% of the trace pairs are distinguishable by the target, whereas for MALCOS-ARM, the validation set (generated using the SMT-backend of Scam-V) is constructed so that 50% of the trace pairs are distinguishable. This ensures that the generated validation sets exercise both leaking and non-leaking behaviors, and, thus, result in informative metrics.

Other side channels: MALCOS generalizes to other observational models (e.g., power or encrypted memory) by integrating domain-specific checkers. For instance, power leaks can be modeled via ISA-level observational equivalence [9], requiring a checker that can test for power leaks and find positive/negative examples.

Leakage contracts and secure programming: Contracts provide the foundations for leak-free software. As shown in [27], ensuring that leakage traces are not dependent on program secrets is sufficient to guarantee the absence of microarchitectural secret-dependent leaks for any CPU satisfying the contract. Since our leakage clauses are of the form *ex* `IF` *pr*, one needs to ensure that (a) the predicate *pr* is always secret-independent and that (b) whenever *pr* holds, then the expression *ex* is also secret-independent.

8 Related work

Over the years, many approaches have been developed to analyze and mitigate hardware side-channels. Here, we summarize works related to leakage contracts and existing techniques to extract them. We also briefly review works on different synthesis approaches, contract synthesis, and automating side-channel analysis.

[5] In §6, "false negatives" are test cases distinguishable by hardware but not by contract traces; thus 100% soundness means capturing all *detected* leaks, not all possible leaks.

Leakage Contracts: Several works aim to formalize microarchitectural features to bridge the hardware-software abstraction gap [13]. Some introduce operational semantics for speculative and out-of-order execution [12, 14, 23, 25, 26, 36, 48, 55], at varying abstraction levels. Others capture microarchitectural side-effects using axiomatic semantics [15, 20, 40]. Our work builds directly on the leakage contract framework [27], which provides a formal basis for relating contracts to concrete leaks in CPUs.

Synthesis of Contracts: The closest work to malcos includes RTL2MμPATH [31], which verifies contracts directly from RTL, and Mohr et al. [39] and LeaSyn [59], who build contracts from manually-specified clauses and RTL tests. ConjunCT [18], VeloCT [19] synthesize coarse-grained contracts for timing attacks, classifying instructions as "safe" or "unsafe" from RTL. We remark that all these tools are template-based, that is, they require user-provided clause templates directly constraining the synthesis search space. In contrast, malcos automatically learns precise, fine-grained contracts for black-box CPUs without templates or RTL access, using counterexample-guided synthesis.

Counterexample-Guided Synthesis: Different works have applied Counterexample-Guided Synthesis (CEGIS) [5] to various tasks, including those related with the hardware-software interface. For instance, Heule et al. [29] and Liu et al. [35] apply CEGIS to automatically synthesize semantics and language semantics. In contrast, PipeSynth [43] uses formal synthesis to generate axioms for microarchitectural memory consistency. Finally, SynthCT [17] synthesizes translations of safe/unsafe instructions. malcos' contract synthesis follows at a high-level the standard CEGIS approach applying example-guided synthesis to leakage contracts expanding the space of examples to pairs of traces instead of single examples.

Automatic Reasoning about Side Channels: Various frameworks automate side-channel analysis, including the fuzzers used in this work, Revizor [44, 45] and Scam-V [11, 41]. Other tools include Osiris [60] (timing-based fuzzing using ISA specs), CheckMate [52] (automated exploit/test generation for hardware), and Xenon [54] (formal verification of constant-time hardware). There are other approaches that leverage symbolic execution, fuzzing, or formal analysis for side-channel vulnerability detection [38, 46, 55, 57, 58, 60, 61].

9 Conclusion

A precise ISA-level leakage contract is critical for secure programming. We introduced a method to extract such contracts from black-box CPUs and validated it with malcos on x86 and ARM CPUs. Our results show that malcos can synthesize precise, sound contracts that capture relevant instruction-level microarchitectural leaks. Furthermore, the results show that our *template-free* approach eliminates the inflexibility of conventional methods, leading to more precise contracts than *template-based* approaches.

References

[1] [n. d.]. Ibex: An embedded 32 bit RISC-V CPU core. https://github.com/lowRISC/ibex

[2] [n. d.]. Ibex RISC-V Core – Multiplier/Divider Block. https://ibex-core.readthedocs.io/en/latest/03_reference/instruction_decode_execute.html#mult-div

[3] Onur Acıiçmez and Çetin Kaya Koç. 2006. Trace-driven Cache Attacks on AES (Short Paper). In *Proceedings of the 8th International Conference on Information and Communications Security* (Raleigh, NC) *(ICICS)*. Springer-Verlag, 112–121.

[4] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, François Dupressoir, and Michael Emmi. 2016. Verifying {Constant-Time} Implementations. In *25th USENIX Security Symposium (USENIX Security 16)*. 53–70.

[5] Rajeev Alur, Rastislav Bodik, Garvit Juniwal, Milo MK Martin, Mukund Raghothaman, Sanjit A Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. 2013. Syntax-guided synthesis. In *2013 Formal Methods in Computer-Aided Design*. IEEE, 1–8.

[6] Ehsan Atoofian and Amirali Baniasadi. 2005. Improving energy-efficiency by bypassing trivial computations. In *19th IEEE International Parallel and Distributed Processing Symposium*. IEEE, 7–pp.

[7] Gilles Barthe, Marcel Böhme, Sunjay Cauligi, Chitchanok Chuengsatiansup, Daniel Genkin, Marco Guarnieri, David Mateos Romero, Peter Schwabe, David Wu, and Yuval Yarom. 2024. Testing side-channel security of cryptographic implementations against future microarchitectures. In *Proceedings of the 31st ACM Conference on Computer and Communications Security (CCS 2024)*. ACM.

[8] Daniel J. Bernstein. 2006. Curve25519: New Diffie-Hellman Speed Records. In *Public Key Cryptography - PKC 2006, 9th International Conference on Theory and Practice of Public-Key Cryptography, New York, NY, USA, April 24-26, 2006, Proceedings (Lecture Notes in Computer Science, Vol. 3958)*, Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin (Eds.). Springer, 207–228. doi:10.1007/11745853_14

[9] Roderick Bloem, Barbara Gigerl, Marc Gourjon, Vedad Hadzic, Stefan Mangard, and Robert Primas. 2022. Power Contracts: Provably Complete Power Leakage Models for Processors. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi (Eds.). ACM, 381–395. doi:10.1145/3548606.3560600

[10] Samira Briongos, Pedro Malagon, Jose M. Moya, and Thomas Eisenbarth. 2020. RELOAD+REFRESH: Abusing Cache Replacement Policies to Perform Stealthy Cache Attacks. In *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 1967–1984. https://www.usenix.org/conference/usenixsecurity20/presentation/briongos

[11] Pablo Buiras, Hamed Nemati, Andreas Lindner, and Roberto Guanciale. 2021. Validation of Side-Channel Models via Observation Refinement. In *MICRO '21: 54th Annual IEEE/ACM International Symposium on Microarchitecture, Greece, October 18-22*. doi:10.1145/3466752.3480130

[12] Sunjay Cauligi, Craig Disselkoen, Klaus v Gleissenthall, Dean Tullsen, Deian Stefan, Tamara Rezk, and Gilles Barthe. 2020. Constant-time foundations for the new spectre era. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. 913–926.

[13] Sunjay Cauligi, Craig Disselkoen, Daniel Moghimi, Gilles Barthe, and Deian Stefan. 2022. SoK: Practical Foundations for Software Spectre Defenses. In *43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22-26, 2022*. IEEE, 666–680. doi:10.1109/SP46214.2022.9833707

[14] Lesly-Ann Daniel, Sébastien Bardin, and Tamara Rezk. 2021. Hunting the haunter-efficient relational symbolic execution for spectre with haunted relse. In *NDSS 2021-Network and Distributed Systems Security*.

[15] Hernán Ponce de León and Johannes Kinder. 2022. Cats vs. Spectre: An Axiomatic Approach to Modeling Speculative Execution Attacks. In *43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22-26, 2022*. IEEE, 235–248. doi:10.1109/SP46214.2022.9833774

[16] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 337–340.

[17] Sushant Dinesh, Grant Garrett-Grossman, and Christopher W Fletcher. 2022. Synthct: Towards portable constant-time code. In *NDSS*.

[18] Sushant Dinesh, Madhusudan Parthasarathy, and Christopher W. Fletcher. 2024. ConjunCT: Learning Inductive Invariants to Prove Unbounded Instruction Safety Against Microarchitectural Timing Attacks. In *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19-23, 2024*. IEEE, 3735–3753. doi:10.1109/SP54263.2024.00180

[19] Sushant Dinesh, Yongye Zhu, and Christopher W Fletcher. 2025. H-Houdini: Scalable Invariant Learning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 603–618.

[20] Craig Disselkoen, Radha Jagadeesan, Alan Jeffrey, and James Riely. 2019. The code that never ran: Modeling attacks on speculative evaluation. In *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1238–1255.

[21] Travis Downs. 2020. Intel's "Zero-Optimization" of Performance Counters. https://travisdowns.github.io/blog/2020/05/13/intel-zero-opt.html. https://travisdowns.github.io/blog/2020/05/13/intel-zero-opt.html Accessed: 2025-04-05.

[22] Goran Doychev, Boris Köpf, Laurent Mauborgne, and Jan Reineke. 2015. Cacheaudit: A tool for the static analysis of cache side channels. *ACM Transactions on information and system security (TISSEC)* 18, 1 (2015), 1–32.

[23] Xaver Fabian, Marco Guarnieri, and Marco Patrignani. 2022. Automatic Detection of Speculative Execution Combinations. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles,*

CA, USA, November 7-11, 2022, Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi (Eds.). ACM, 965–978. doi:10.1145/3548606.3560555

[24] Daniel Gruss, Clémentine Maurice, Klaus Wagner, and Stefan Mangard. 2016. Flush+Flush: a fast and stealthy cache attack. In *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 279–299.

[25] Roberto Guanciale, Musard Balliu, and Mads Dam. 2020. InSpectre: Breaking and Fixing Microarchitectural Vulnerabilities by Formal Analysis. In *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna (Eds.). ACM, 1853–1869. doi:10.1145/3372297.3417246

[26] Marco Guarnieri, Boris Köpf, José F Morales, Jan Reineke, and Andrés Sánchez. 2020. Spectector: Principled detection of speculative information flows. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1–19.

[27] Marco Guarnieri, Boris Köpf, Jan Reineke, and Pepe Vila. 2021. Hardware-software contracts for secure speculation. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1868–1883.

[28] David Gullasch, Endre Bangerter, and Stephan Krenn. 2011. Cache games–bringing access-based cache attacks on AES to practice. In *2011 IEEE Symposium on Security and Privacy*. IEEE, 490–505.

[29] Stefan Heule, Eric Schkufza, Rahul Sharma, and Alex Aiken. 2016. Stratified synthesis: automatically learning the x86-64 instruction set. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 237–250.

[30] Jana Hofmann, Emanuele Vannacci, Cédric Fournet, Boris Köpf, and Oleksii Oleksenko. 2023. Speculation at Fault: Modeling and Testing Microarchitectural Leakage of CPU Exceptions. In *32nd USENIX Security Symposium (USENIX Security 23)*. 7143–7160.

[31] Yao Hsiao, Nikos Nikoleris, Artem Khyzha, Dominic P Mulligan, Gustavo Petri, Christopher W Fletcher, and Caroline Trippel. 2024. μPATH: Multi-μPATH Synthesis with Applications to Hardware Security Verification. *arXiv preprint arXiv:2409.19478* (2024).

[32] Intel. [n. d.]. Speculative Store Bypass / CVE-2018-3639 / INTEL-SA-00115. https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/advisory-guidance/speculative-store-bypass.html.

[33] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. 2019. Spectre Attacks: Exploiting Speculative Execution. In *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*. IEEE, 1–19. doi:10.1109/SP.2019.00002

[34] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, and Mike Hamburg. 2018. Meltdown. *arXiv preprint arXiv:1801.01207* (2018).

[35] Jiangyi Liu, Charlie Murphy, Anvay Grover, Keith JC Johnson, Thomas Reps, and Loris D'Antoni. 2024. Synthesizing Formal Semantics from Executable Interpreters. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024), 362–388.

[36] Ross Mcilroy, Jaroslav Sevcik, Tobias Tebbi, Ben L Titzer, and Toon Verwaest. 2019. Spectre is here to stay: An analysis of side-channels and speculative execution. *arXiv preprint arXiv:1902.05178* (2019).

[37] Microsoft. 2024. Configuration File. https://microsoft.github.io/sca-fuzzer/user/config/#executor-configuration Accessed: 2025-06-16.

[38] Daniel Moghimi, Moritz Lipp, Berk Sunar, and Michael Schwarz. 2020. Medusa: Microarchitectural Data Leakage via Automated Attack Synthesis. In *29th USENIX Security Symposium, USENIX Security 2020, August 12-14, 2020*, Srdjan Capkun and Franziska Roesner (Eds.). USENIX Association, 1427–1444. https://www.usenix.org/conference/usenixsecurity20/presentation/moghimi-medusa

[39] Gideon Mohr, Marco Guarnieri, and Jan Reineke. 2024. Synthesizing Hardware-Software Leakage Contracts for RISC-V Open-Source Processors. In *Design, Automation & Test in Europe Conference & Exhibition, DATE 2024, Valencia, Spain, March 25-27, 2024*. IEEE, 1–6. doi:10.23919/DATE58400.2024.10546681

[40] Nicholas Mosier, Hanna Lachnitt, Hamed Nemati, and Caroline Trippel. 2022. Axiomatic hardware-software contracts for security. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 72–86.

[41] Hamed Nemati, Pablo Buiras, Andreas Lindner, Roberto Guanciale, and Swen Jacobs. 2020. Validation of Abstract Side-Channel Models for Computer Architectures. In *Computer Aided Verification - 32nd International Conference, CAV 2020 Los Angeles, CA, USA, July 21-24*. doi:10.1007/978-3-030-53288-8_12

[42] Michael Neve and Jean-Pierre Seifert. 2007. Advances on Access-driven Cache Attacks on AES. In *Proceedings of the 13th International Conference on Selected Areas in Cryptography* (Montreal, Canada) *(SAC'06)*. Springer-Verlag, 147–162.

[43] Chase Norman, Adwait Godbole, and Yatin A Manerkar. 2023. PipeSynth: Automated Synthesis of Microarchitectural Axioms for Memory Consistency. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 513–527.

[44] Oleksii Oleksenko, Christof Fetzer, Boris Köpf, and Mark Silberstein. 2022. Revizor: Testing black-box CPUs against speculation contracts. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 226–239.

[45] Oleksii Oleksenko, Marco Guarnieri, Boris Köpf, and Mark Silberstein. 2023. Hide and Seek with Spectres: Efficient discovery of speculative information leaks with random testing. *arXiv preprint arXiv:2301.07642* (2023).

[46] Oleksii Oleksenko, Bohdan Trach, Mark Silberstein, and Christof Fetzer. 2020. SpecFuzz: Bringing Spectre-type vulnerabilities to the surface. In *29th USENIX Security Symposium (USENIX Security 20)*.

[47] Dag Arne Osvik, Adi Shamir, and Eran Tromer. 2006. Cache Attacks and Countermeasures: The Case of AES. In *Proceedings of the 2006 The Cryptographers' Track at the RSA Conference on Topics in Cryptology* (San Jose, CA) *(CT-RSA'06)*. Springer-Verlag, Berlin, Heidelberg, 1–20. https://doi.org/10.1007/11605805_1

[48] Marco Patrignani and Marco Guarnieri. 2021. Exorcising Spectres with Secure Compilers. In *Proceedings of the 28th ACM Conference on Computer and Communications Security (CCS 2021)*. ACM.

[49] Colin Percival. 2005. Cache Missing for Fun and Profit. In *In Proc. of BSDCan 2005*.

[50] Robin Leander Schröder, Stefan Gast, and Qian Guo. 2024. Divide and Surrender: Exploiting Variable Division Instruction Timing in {HQC} Key Recovery Attacks. In *33rd USENIX Security Symposium (USENIX Security 24)*. 6669–6686.

[51] Emina Torlak and Rastislav Bodik. 2013. Growing solver-aided languages with Rosette. In *Proceedings of the 2013 ACM international symposium on New ideas, new paradigms, and reflections on programming & software*. 135–152.

[52] Caroline Trippel, Daniel Lustig, and Margaret Martonosi. 2018. CheckMate: Automated Synthesis of Hardware Exploits and Security Litmus Tests. In *Proceedings of the 51st Annual IEEE/ACM International Symposium on Microarchitecture* (Fukuoka, Japan) *(MICRO-51)*. IEEE Press, 947–960. doi:10.1109/MICRO.2018.00081

[53] Yukiyasu Tsunoo, Teruo Saito, Tomoyasu Suzaki, and Maki Shigeri. 2003. Cryptanalysis of DES Implemented on Computers with Cache. In *Proceedings of the Workshop on Cryptographic Hardware and Embedded Systems (CHES'03, LNCS)*. Springer, 62–76.

[54] Klaus v. Gleissenthall, Rami Gökhan Kıcı, Deian Stefan, and Ranjit Jhala. 2021. Solver-aided constant-time hardware verification. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 429–444.

[55] Marco Vassena, Craig Disselkoen, Klaus von Gleissenthall, Sunjay Cauligi, Rami Gökhan Kıcı, Ranjit Jhala, Dean Tullsen, and Deian Stefan. 2021. Automatically eliminating speculative leaks from cryptographic code with blade. *Proceedings of the ACM on Programming Languages* 5, POPL (2021), 1–30.

[56] Jose Rodrigo Sanchez Vicarte, Pradyumna Shome, Nandeeka Nayak, Caroline Trippel, Adam Morrison, David Kohlbrenner, and Christopher W Fletcher. 2021. Opening pandora's box: A systematic study of new ways microarchitecture can leak private data. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 347–360.

[57] Guanhua Wang, Sudipta Chattopadhyay, Arnab Kumar Biswas, Tulika Mitra, and Abhik Roychoudhury. 2020. KLEESpectre: Detecting information leakage through speculative cache attacks via symbolic execution. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 29, 3 (2020), 1–31.

[58] Zilong Wang, Gideon Mohr, Klaus von Gleissenthall, Jan Reineke, and Marco Guarnieri. 2023. Specification and Verification of Side-channel Security for Open-source Processors via Leakage Contracts. In *Proceedings of the 30th ACM Conference on Computer and Communications Security (CCS 2023)*. ACM.

[59] Zilong Wang, Gideon Mohr, Klaus von Gleissenthall, Jan Reineke, and Marco Guarnieri. 2025. Synthesis of Sound and Precise Leakage Contracts for Open-Source RISC-V Processors. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. 111–125.

[60] Daniel Weber, Ahmad Ibrahim, Hamed Nemati, Michael Schwarz, and Christian Rossow. 2021. Osiris: Automated Discovery of Microarchitectural Side Channels. *CoRR* abs/2106.03470 (2021). arXiv:2106.03470 https://arxiv.org/abs/2106.03470

[61] Yuheng Yang, Thomas Bourgeat, Stella Lau, and Mengjia Yan. 2023. Pensieve: Microarchitectural modeling for security evaluation. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–15.

[62] Yuval Yarom and Katrina Falkner. 2014. Flush+Reload: a high resolution, low noise, L3 cache side-channel attack. In *Proceedings of the 23rd USENIX Conference on Security Symposium*. 719–732.